

HackerRank Algorithmic Problem-Solving Portfolio

Activity 8 | 3rd Semester B.Tech, Computer Science and Engineering

Student: Sivarankan T **Roll number:** DCET2600466

Repository and profile

GitHub repository: <https://github.com/shivasivs/HackerRank-3rdSem-Portfolio>

HackerRank profile: Account created; add your public profile URL here before submission.

Problem set and complexity

Problem	Method	Time	Extra space
Diagonal Difference	Traverse both diagonals in one pass	$O(N)$	$O(1)$
Dynamic Array	XOR index and sequence lookup	$O(N + Q)$	$O(N)$
Time Conversion	Adjust hour for AM/PM boundaries	$O(1)$	$O(1)$
Compare the Triplets	Compare three corresponding scores	$O(1)$	$O(1)$
Sparse Arrays	Count strings, then answer lookups	$O(N + Q)^*$	$O(N)$

*For bounded-length strings; more precisely, expected $O(\text{total characters})$ across stored strings and queries.

Evidence status

The local solutions and analysis are prepared. HackerRank Accepted results, a 3-star badge, and profile details must be added only after they are earned and captured from the student account. This report is a draft until those account details are filled in.

Reflection on Algorithmic Optimization

Draft reflection - personalize after attempting the challenges on HackerRank

Working through these problems highlights how choosing the right representation can make a solution both simpler and faster. For Diagonal Difference, I only need to visit the two diagonal entries in each row, instead of scanning every matrix cell. Dynamic Array depends on applying the XOR-based index exactly as specified and keeping each sequence separate. Time Conversion is a useful reminder to handle boundary values explicitly: midnight and noon do not follow the same adjustment as other hours. For Compare the Triplets, the fixed input size makes a direct comparison easy to reason about. Sparse Arrays benefits from a frequency map, which avoids rescanning every input string for every query. Across the set, I found that understanding the input constraints helps decide whether constant extra space, a list of sequences, or a hash map is appropriate. I also need to check edge cases, especially equal scores, AM/PM boundaries, missing strings, and queries after the sequence has changed. Before submitting, I will run each solution against the sample cases, compare the returned format with HackerRank's required function signature, and review the time and space analysis. I will update this reflection with the specific challenge that took me longest and what I learned from my actual accepted submissions.

Reflection draft: 206 words. Replace the final sentence with your own experience before submitting.

Submission evidence checklist

- ☐ Five genuine Accepted submission captures saved in the repository
- ☐ Public HackerRank profile URL added above
- ☐ At least one genuine 3-star badge earned and captured
- ☐ GitHub repository made public and the live URL checked
- ☐ Reflection personalized to match the work actually completed